

DocAgent

Intelligenter Dokumenten-Assistent mit RAG, FAISS und lokalen LLMs

Version 0.8 Beta (Stand: 17.06.2026)

Aktueller Entwicklungsstand: Phase 3

Projektleitung und Entwicklung

Klaus Kremer
Jahnstraße 91
40215 Düsseldorf
Email: kl.kremer@web.de

Projektziel

DocAgent ist ein lokaler KI-gestützter Dokumentenassistent.

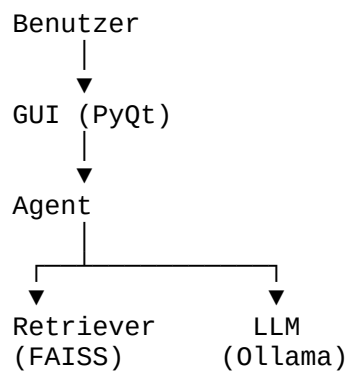
Das System ermöglicht:

- Dokumente verwalten
- Dokumente indizieren
- Informationen aus Dokumenten suchen
- Fragen zu Dokumenten beantworten
- Dokumente zusammenfassen
- Quellen nachvollziehbar anzeigen

Der Fokus liegt auf:

- lokaler Ausführung
- Datenschutz
- Nachvollziehbarkeit
- einfacher Bedienung

Architekturübersicht



Hauptkomponenten

GUI

Datei:

`ui/main_window.py`

Aufgaben:

- Chatoberfläche
- Dokumentenverwaltung
- Quellenanzeige
- Statusmeldungen
- Thinking-Animation
- Dateiauswahl
- Dokument löschen
- Dokument hinzufügen

Agent

Datei:

`core/agent.py`

Aufgaben:

- Intent-Erkennung
- Query-Rewriting
- Retrieval-Steuerung
- Prompt-Erstellung
- Antwortvalidierung
- Dokumentzusammenfassungen

Retriever

Datei:

`core/retriever.py`

Aufgaben:

- Embedding-Erzeugung
- FAISS-Suche
- Re-Ranking
- Score-Berechnung

Ollama Client

Datei:

`core/ollama_client.py`

Aufgaben:

- Kommunikation mit lokalem LLM
- Prompt-Übergabe
- Antwortgenerierung

Dokumentenpipeline

Unterstützte Formate

Aktuell:

PDF
DOCX
ODT
TXT

Optional vorbereitet:

DOC

Einlesen

PDF

Bibliothek:

Pypdf

DOCX

Bibliothek:

Python-docx

ODT

Bibliothek:

odfpy

TXT

Standard-Dateizugriff

Chunking

Aktuelle Strategie:

Absatzbasiertes Chunking

Parameter:

`max_length = 500`

Ziel:

- semantische Einheiten erhalten
- Kontextverluste minimieren

Embeddings

Modell:

`intfloat/multilingual-e5-base`

Eigenschaften:

- mehrsprachig
- gute deutsche Ergebnisse
- lokal ausführbar

Vektorindex

Bibliothek:

FAISS

Index:

`faiss.IndexFlatIP`

Similarity:

Cosine Similarity

über:

`faiss.normalize_L2()`

Re-Ranking

Modell:

cross-encoder/ms-marco-MiniLM-L-6-v2

Zweck:

Verbesserung der Trefferqualität nach der FAISS-Suche.

Retrieval-Prozess

1

Benutzerfrage

Wann wurde die Versicherung gekündigt?

2

Intent Detection

Kategorien:

FACT
SUMMARY
LIST

3

Query Rewrite

Optimierung der Suchanfrage.

4

FAISS Retrieval

Top-K Kandidaten:

FACT → 3
SUMMARY → 5
LIST → 10

5

Cross Encoder Re-Ranking

Bewertung der Kandidaten.

6

Score-Berechnung

Kombination aus:

Rerank Score
FAISS Score
Keyword Boost

7

Prompt-Erstellung

Kontext wird aufgebaut:

[SOURCE: datei.pdf]

Text...

8

LLM-Antwort

Generierung über Ollama.

9

Verifikation

Antwort wird geprüft.

Bei Problemen:

Fallback Prompt

Dokumentenverwaltung

Speicherort

Dokumente:

data/docs

Metadaten

documents.pkl

Enthält:

```
{  
    "content": "...",  
    "source": "...",  
}
```

Vektorindex

data/faiss.index

Incremental Indexing

Ziel:

Neue Dokumente hinzufügen ohne vollständigen Rebuild.

Ablauf:

```
Datei hinzufügen  
↓  
Hash prüfen  
↓  
Chunking  
↓  
Embeddings erzeugen  
↓  
FAISS.add()
```

Dubletten-Erkennung

Methode:

SHA256 Hash

Anwendungsfall:

Dokument bereits vorhanden

→ Import wird verhindert.

Dokument löschen

Aktueller Ansatz:

Datei löschen

↓

documents.pkl aktualisieren

↓

FAISS Rebuild aus vorhandenen Chunks

Vorteile:

- robust
- einfach
- keine komplexen IDs nötig

Quellenanzeige

Jede Antwort enthält:

Dateiname

Score

Textvorschau

Beispiel:

 ohl_co2_160326.pdf

Score: 1.84

Klaus Kremer · Jahnstraße 91 ·
40215 Düsseldorf ...

Dokumentzusammenfassung

Neue Funktion.

Beispiel:

Fasse das Dokument `vertrag.pdf` zusammen

Ablauf:

Dokument finden

↓

Alle Chunks laden

↓

LLM-Zusammenfassung erzeugen

GUI-Funktionen

Chat

Fragen stellen

Dokumentliste

Anzeige aller Dokumente.

Dokument hinzufügen

Dateiauswahl.

Dokument löschen

Mit Sicherheitsabfrage.

Quellenfenster

Anzeige:

- Quelle
- Score
- Vorschau

Dokument öffnen

Klick auf Quelle öffnet Originaldokument.

Thinking Animation

Statusanzeige:

Agent denkt.
Agent denkt..
Agent denkt...

Bekannte Optimierungen

Absender / Empfänger

Systemprompt erweitert:

Bei Briefen:

- Absender und Empfänger unterscheiden
- Keine Annahmen treffen

Ergebnis:

deutlich bessere Antworten.

Aktuelle Stärken

- vollständig lokal
- gute deutsche Dokumentverarbeitung
- nachvollziehbare Quellen
- schnelle Suche
- inkrementelles Hinzufügen
- Dokumentzusammenfassungen
- moderne GUI

Geplante Erweiterungen

Dokumentanalyse

Personen
Adressen
Termine
Beträge
Fristen

Dokumenttypen

Vertrag
Rechnung
Brief
Rezept

Dokumentvergleich

Vergleiche Dokument A mit Dokument B

Erweiterte Metadaten

Dateigröße
Indexdatum
Chunkanzahl
Hash

Technologiestack

Python 3.11
PyQt6
FAISS
SentenceTransformers
PyTorch
Ollama
pypdf
python-docx
odfpy
NumPy
pickle

Projektstatus

DocAgent befindet sich aktuell in einem stabilen Entwicklungsstand mit funktionierender Dokumentverwaltung, Retrieval-Augmented Generation (RAG), Quellenverfolgung und Dokumentzusammenfassung.

Der Schwerpunkt der nächsten Entwicklungsphase liegt auf der Verbesserung der Dokumentanalyse, der Retrieval-Qualität und der strukturierten Informationsgewinnung aus Dokumenten.